

Estimation of Distribution using Population Queue based Variational Autoencoders

Sourodeep Bhattacharjee
School of Computer Science
University of Windsor
Windsor, Canada
bhattach1@uwindsor.ca

Robin Gras
School of Computer Science, Department of Biological Sciences
Great Lakes Institute for Environmental Research
University of Windsor
Windsor, Canada
rgras@uwindsor.ca

Abstract—We present a new Estimation of Distribution algorithms (EDA) based on two novel Variational Autoencoders generative model building algorithms. The first method, Variational Autoencoder with Population Queue (VAE-EDA-Q), employs a queue of historical populations, which is updated at each iteration of EDA in order to smooth the data generation process. The second method uses Adaptive Variance Scaling (AVS) with VAE-EDA-Q to dynamically update the variance at which the probabilistic model is sampled. The results obtained prove our methods to be significantly more computationally efficient than state-of-the-art algorithms and perform significantly less number of fitness evaluations when tested on benchmark problems such as Trap-k and NK Landscapes. Moreover, we report results of applying our approach successfully to highly complex problems such as Trap 11, Trap 13, and NK Landscapes with neighborhood size $K = 8$ and $K = 10$.

Index Terms—Estimation of Distribution Algorithms, Variational Autoencoders, Machine Learning, Combinatorial Optimization

I. INTRODUCTION

Estimation of Distribution Algorithms (EDA) [1]–[4] are population-based stochastic combinatorial optimization algorithms that attempt to explore a solution space to find an optimum solution by creating probabilistic models of promising solutions and creating new samples from the probabilistic model. The algorithms typically try to generalize from the current solution space sample (current population) to build a probabilistic model; this model is then sampled to create the next generation of solutions that are evaluated with respect to the function to optimize (the fitness function). The best new solutions are selected and are recombined with the parent population (see figure 1 adapted from [5]). The population model is improved at every iteration of EDA to yield the next generation of solutions, that could be potentially closer to the global optima.

In recent studies, Machine Learning techniques have been employed to build population models with considerable success. Restricted Boltzmann Machine (RBM) [6] comprising of a stochastic neural network was used as population model [7] which had considerably less computational time requirement than BOA. In another work, the authors implemented Deep Boltzmann Machine (DBM) to generate the next batches of solutions [8]. Autoencoders [9], [10] have also been used to model the population of promising solutions [11].

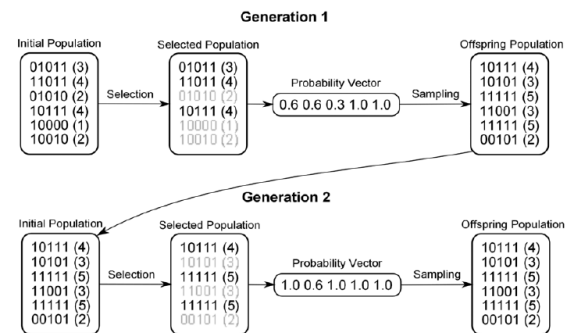


Fig. 1. EDA Procedure: The initial population is assigned a fitness value by application of a fitness function on the candidate and selection step is applied to pick the fittest candidates out of the parent pool. A simple probabilistic model is created from the fittest parents which is then sampled to produce next generation of candidates. [5]

In this paper we propose a novel extension for EDA that uses Variational Autoencoders [12] - Variational Autoencoder EDA (VAE-EDA) [13]. Variational autoencoders are expected to yield models that explore the solution space more optimally as they model a probabilistic distribution of the latent space (its average and standard deviation on the assumption that the latent variables follow a Gaussian distribution) that would better capture the properties of the objective function, which allows for the smoothing of the data generation process [13]. The extension of VAE-EDA (VAE-EDA-Q) proposed in this paper smooths the update of the VAE-EDA model to avoid premature convergence to a sub-optimal region of the solution-space, by learning the model on a set of candidate solutions generated on a range of previous generations (modeled as a queue of populations) of the VAE-EDA process. Another extension proposed in this work uses Adaptive Variance Scaling (AVS) [14] to dynamically control the rate of exploration of the latent space by using a coefficient multiplied to the variance of sampling.

II. BACKGROUND AND RELATED WORK

A. Bayesian optimization algorithm and improvements

Bayesian optimization algorithm (BOA) [15] [16] is a state-of-the-art EDA capable of tackling many difficult optimization

problems. It uses Bayesian networks to model the population and hence solves many problems that are difficult to decompose into separable subproblems. The possible configuration of a Bayesian Network grows exponentially with the number of decision variables. Hence, a greedy heuristic is used to build a network from an empty network consisting of nodes only. Some real world problems are hierarchically decomposable, and hBOA [17] algorithm tackles such problems by creating compact Bayesian networks with local structures to allow complex networks to be learned [5]. Additionally, hBOA uses restricted tournament replacement, which is a niching technique that attempts to match similar solutions against each other rather than competing dissimilar solutions.

Constrained Model Search Space BOA (CMSS-BOA) [18], on the other hand successfully improves on the computational efficiency of BOA by constraining the search space using a heuristic called max-min parent children (MMPC) [19] and then performing hill-climbing on the search space. CMSS-BOA was compared to BOA on benchmark problems such as OneMax and concatenated k-trap function. CMSS-BOA was found, in most cases, to be able to converge earlier than BOA. However the average fitness of BOA population was slightly higher than CMSS-BOA.

B. Deep Boltzmann Machine based EDA

Deep Boltzmann Machines (DBM) [20] are deep neural networks consisting of multiple layers of hidden neurons, and, hence, can be used to capture the model of the population at increasing layers of abstraction. Each hidden layer provides one additional layer of abstraction capable of representing patterns or features in the data. The authors in [8] used DBM with EDA based on the promise shown by the results in RBM-EDA and based on the success of deep learning models. The authors found that while DBM-EDA is computationally less expensive than BOA, the quality of the model was not as good as BOA in case of multi modal problems beyond trap-5. The authors state that, given the efforts needed to train, it is not feasible to be used in a noisy training set. DBM-EDA could not find the global optimum in complex problems when allotted same population size as BOA.

C. Autoencoders and Denoising Autoencoder EDA

Autoencoders [9], [10], [21] are neural networks that learn an abstract representation of the input with an encoder layer and then perform a reconstruction step using a decoder to convert the representation to an output that approximates the distribution of training examples. Autoencoders have been used as generative modeling techniques in dimensionality reduction or for feature learning [22].

One of the methods used to prevent such over-fitting is to use Denoising Autoencoders (DAE) [23] where the input or the training sample is intentionally corrupted with a certain type of noise (uniform, Gaussian etc.), and the denoising autoencoder is trained to filter out the noise in a denoising layer and thereafter reconstruct the output from a “cleaned” input. This makes denoising autoencoders a robust filter to noise in the

training samples. Denoising Autoencoder EDA (DAE-EDA) [11] uses denoising autoencoders as generative models for EDA. DAE-EDA was tested on concatenated deceptive traps, NK Landscapes, and HIFF functions, and its performance was compared to BOA and RBM-EDA. DAE-EDA was found to be much faster than BOA and RBM-EDA however at the cost of reduced model quality.

III. VARIATIONAL AUTOENCODER EDA

A. Variational Autoencoders

Variational Autoencoders (VAE) [12] [24] improve upon traditional autoencoders in their explorative capabilities by generating data from continuous latent spaces representation which allows for smooth interpolation and randomized sampling by expanding the latent search space horizon beyond what the training samples represent. While traditional autoencoders are able to introduce minor random variations in the input provided, VAEs allow for a greater degree of control of the direction and magnitude of the variation produced in the output by the hidden model [25]. Autoencoders described in the previous section differ from VAE in that standard autoencoders do not have a probabilistic basis while VAEs allow stochastic generative modeling with tunable probability parameters [26]. We have adapted the following description of VAE from [25].

In [12] and [25] the authors define VAE by considering z as a vector of latent variables in $\mathcal{Z}$ which form a high dimensional space (see Encoder half of figure 2). The z can be sampled using some density function $P(z)$ defined over $\mathcal{Z}$. Equation 1 aims at maximizing the probability of X by optimizing θ (X is a data point in our dataset, and θ is a parameter vector in space Θ); where z is sampled with a high probability from $P(z)$.

$$P(X) = \text{Maximize}(\int P(X|z; \theta)P(z)dz) \quad (1)$$

Hence in VAE the objective becomes to model the probabilistic distribution of parameters of the latent space instead of fixed values for them [26]. However, the integral computation is intractable. Some approximations are therefore needed to estimate this distribution. VAE use an approximation $Q(z|X)$, modeled by the encoder, of the posterior distribution $P(z|X)$ and an approximation of $P(X|z)$, modeled by the decoder. Some common choices of the distribution models used in VAEs are Bernoulli Distribution or Multivariate Gaussian distribution. Equation 2 gives the probability model for Gaussian distribution having a mean of $f(z; \theta)$, with I being the covariance multiplied by some scalar hyperparameter σ and f is the decoder network.

$$P(X|z; \theta) = \mathcal{N}(X|f(z; \theta), \sigma^2 * I) \quad (2)$$

The values sampled from the model for z need to contain useful information from $P(X|z)$ to produce an output similar to desired target X . Therefore the function $Q(z|X)$ is learned (encoder) in order that the distribution over z can be later used to produce X .

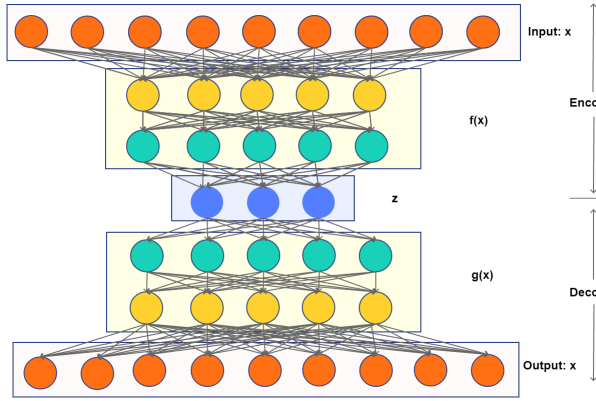


Fig. 2. Variational Autoencoder structure showing the parent vectors on top layer mapped to higher dimensional spaces in encoder $f(x)$. The z layer provides the probability distribution sampling function which is sampled by the decoding layers $g(x)$ to yield the output in real dimension at the bottom layer.

B. Variational Autoencoder EDA (VAE-EDA)

Authors in [13] used VAE as a generative modeling technique to be used as the model for EDA (VAE-EDA). The motivation is that VAEs have continuous latent spaces and allow for easy interpolation and random sampling. To clarify, if the input space has gaps or discontinuities, standard EDAs have a hard time dealing with the corresponding latent space. With VAE-EDA, a randomized population is initially generated from which individuals are selected by using Tournament Selection to find and select the subset of fittest individuals. From these fittest individuals, a Gaussian probabilistic VAE model is learned, that is then sampled to derive offspring candidates for the next generation.

C. Variational Autoencoder EDA with Population Queue (VAE-EDA-Q)

We propose an advanced version of VAE-EDA algorithm in algorithm 1, in which we attempt to control the convergence of our population by using a strategy in which the VAE-EDA is updated using a sample built from a sliding window of historical populations. We refer to this algorithm as Variational Autoencoder EDA with Population Queue (VAE-EDA-Q)¹. The VAE-EDA-Q model is updated based on fittest candidates from the sampled population history instead of immediate parent population. We perform tournament selection on candidates from the history to create a new population to update the VAE-EDA-Q model. The latent model of the VAE-EDA-Q is sampled at every time step and a new population is built by combining the offspring with the fittest candidates in the parent population for the preceding time step. The VAE-EDA-Q is expected to have a greater degree of control over the diversity of the solutions produced. The size of the queue allows us to control this diversity - if the value is too low, the model produces too diverse solutions. And if the value is on the higher side, then the model does not explore as much

as desired. The choice of the value for which this parameter is set depends on the difficulty and the nature of the problem to be solved. In our study, the update time step was empirically chosen as 10 to find a value that balances speed of convergence and desirable amount of population diversity for the problems in the scope of this paper.

D. Variational Autoencoder EDA with Population Queue and Adaptive Variance Scaling (VAE-EDA-Q AVS)

In order to remedy the problem of diversity loss (premature loss of variance in the population), a methodology named Adaptive Variance Scaling (AVS) was suggested in [14] [27] that helped scale the variance beyond the maximum likelihood estimate of the model generated from the parent population. In AVS, a coefficient is used to scale the variance of sampling, which is known as variance multiplier c^{AVS} . If the variance of the model is Σ then when sampling the model is sampled using a variance of $c^{AVS}\Sigma$. If at each iteration we find an improvement in the best fitness, then the c^{AVS} is multiplied by $\eta^{INC} > 1$ to help enlarge the variance and increase the exploration of the search space.

If no improvement is observed in fitness values, the variance multiplier is multiplied by a factor of $\eta^{DEC} \in [0, 1]$, which helps in reducing the range of exploration. Generally, in order to maintain symmetry in scaling, $\eta^{INC} = 1/\eta^{DEC}$. We have applied the AVS method to the standard deviation vectors of VAE-EDA-Q at the layer of hidden neurons that model the latent space (z layer in figure 2) to dynamically control the amount of exploitation and exploration at the sampling step of each iteration. We will refer to this algorithm as VAE-EDA-Q AVS¹. VAE-EDA-Q AVS was able to solve benchmark problems of even greater difficulty than VAE-EDA-Q such as Trap 11 and 13 as well as NK Landscapes of up to epistasis (k) 10.

IV. EXPERIMENTS

A. Test Problems

We have evaluated VAE-EDA-Q and VAE-EDA-Q AVS on concatenated deceptive trap functions of order 5, 7 and 9, as well as NK Landscapes (with the epistasis k varying from 4 to 6), which are considered benchmark test problems [9]. Additionally, VAE-EDA-Q AVS was also tested on Trap Functions of order 11 and 13 and also on NK Landscape of $k = 6$ to $k = 8$. The difficulty of these problems increased linearly with the problem size - given by the number of decision variables, as well as exponentially with the degree of dependencies between parameters (for example the order of the trap). Therefore, the main driving factor of complexity for these problems is the degree of dependencies.

In concatenated traps [28] of order k (referred to as Trap-K), a solution vector x is divided in subsets of size k , where each subset is used to create a deceptive trap. Within each subset, the variables are dependent on each other but independent of the variables in other subsets, so the problem is decomposable

¹Source code is available at <https://github.com/sourodeep/vaedeaq>

Algorithm 1 Variational Autoencoder EDA with Population Queue - VAE-EDA-Q

```
i ← 0
QueueSize ← 10
PopulationHistory[] ← null                                ▷ Define a Queue to store populations from different iterations
Generate Initial Population P(0)                          ▷ Assume n to be population size
PopulationHistory[i].PUSH(P(0))                          ▷ Insert initial population into the queue of populations
while Global Optimum not found OR Current Iteration j Max Iteration do
    Select promising solutions N(i) from P(i) using TournamentSelection
    if i ≥ QueueSize then
        PopulationHistory.POP()                            ▷ Remove oldest Population from head of the Queue
    end if
    SampledPopulationHistory ← TournamentSelection(PopulationHistory[])    ▷ Selecting n candidates
    Build/Update Variational Autoencoder Model M(i) from SampledPopulationHistory
    Sample M(i) to produce next generation O(i)
    P(i + 1) ← N(i) + O(i)
    PopulationHistory.PUSH(P(i + 1))                      ▷ Insert newly generated population into the queue
    i ← i + 1
end while
```

into independent sub-problems. Hence, the fitness of each subset contributes to the global fitness as a sum of fitness of individual traps.

Assume p to be an assignment of k values (0 or 1) to an arbitrary subset of variables, then the fitness contribution for that subset is given by Equation 7. The fitness of the trap increases with increasing zeros, unless the assignment are all ones. The deception lies in the fact that with increasing numbers of 1 in a subset, the fitness decreases even though the solution gets closer to global optimum (consisting of an assignment of 1 for all variables). The size of the subset or trap given by k increases the complexity of the problem with increasing value for k .

$$f(a) = \begin{cases} k & \text{if } \sum_i^{i+k-1} p_i = k \\ k - (\sum p_i + 1) & \text{if otherwise} \end{cases} \quad (3)$$

NK Landscapes [29] are functions defined by two parameters: the total number of binary variables in the problem (N) and the size of neighborhood for each variable (K). The fitness function of each bit is provided by a lookup table generated randomly beforehand and determines the fitness of the bit and its neighbors. For example we can consider a problem of $N = 4$ and $K = 1$, meaning the problem contains 4 variables in total and the fitness of the assignment of all variables $f(n)$ is composed of the sum of the fitness of sub-problems f_i^N , one sub-problem for each variable. The fitness of one variable depends on its value and the value of its next k variables (one in our example since $K=1$).

In NK Landscapes the total fitness of a solution $f(n)$ is the sum of all fitness components (subsets of a bit or variable and its k neighbors). Formally equation 4 [7] shows how the total fitness $f(n)$ is governed by the sum of individual fitness functions f_i^N . f_i^N evaluates fitness of a sub-problem based on the assignment of the first variable x_i as well as its k successors x_{i1} to x_{iK} . Therefore, the value each

decision variable x_i affects the fitness of $K+1$ sub-functions. Thus, each decision variable assignment impacts multiple fitness components due to this overlap among neighboring variables. These dependencies make it impossible to separate and independently solve the sub-problems.

$$f(x) = 1/N \sum_{i=1}^N f_i^N(x_i, x_{i1}, \dots, x_{iK}) \quad (4)$$

In most cases NK Landscapes are NP-Complete for $k > 1$, and poses a challenging optimization problem [30]. In order to compare VAE-EDA, VAE-EDA-Q and VAE-EDA-Q AVS with other state-of-the-art algorithms (BM-EDA [7] and DAE-EDA [11]) we use similar NK Landscapes with known optima as in [7], [11], [30].

B. Experimental Setup

We have compared VAE-EDA [13], VAE-EDA-Q, VAE-EDA-Q AVS, BOA [15], hBOA [17], BM-EDA [7], CMSS-BOA [18], DBM-EDA [20] and DAE-EDA [11], on Trap Functions and NK Landscapes of varying complexity. For each algorithm and problem, we have reported results for an average of 30 runs, which were allowed to run until global optimum was found. For consistency of the results, all algorithms used tournament selection (size 2) without replacement (same as reported in experiments for DBM-EDA and DAE-EDA). We have used the results mentioned by the authors in the original papers for BM-EDA and DAE-EDA in our discussions. For Trap 5 we have compared VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, BOA, hBOA, DAE-EDA, DBM-EDA and CMSS-BOA and reported the execution times in seconds (log scale) and the number of unique fitness evaluations. For Trap 7 and 9 we have compared VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, BOA, hBOA and CMSS-BOA and reported the execution times and total number of fitness evaluations. DAE-EDA and DBM-EDA could not be

compared for Trap 7 and 9 as the results for the algorithm were not reported by the authors as their code was not available.

For NK Landscapes (up to $K = 6$) we compared VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, BOA, hBOA, DBM-EDA, CMSS-BOA and DAE-EDA and reported the execution times and the total number of unique fitness evaluations. Some of the algorithms could not produce results in a comparable time frame or number of generations for the particular test problem, and, hence could not be included in the results. For Traps of 11 and 13, and also for NK Landscape of $K = 8$ and $K = 10$, we have compared only VAE-EDA-Q AVS, BOA and CMSS-BOA as the other algorithms were either not able to solve these problems or the results were not available from other published research. To the best of our knowledge, this paper is the first to present successful results on Trap 11 and 13 and on NK-landscape with $K=10$. All experiments were performed on a high performance cluster computing platform provided by SHARCNET (<https://www.sharcnet.ca>) where base compute node configuration was Intel E5-2683 v4 (Broadwell) 2.1 GHz with 32 cores, 128 GB of memory. For Trap 5 and NK Landscape ($K < 8$) we have used results reported in previously published research. For all other test problems, the tests were performed on the same system configuration.

C. Implementation

For both the algorithms proposed in this paper, VAE-EDA-Q¹ and VAE-EDA-Q AVS, Nesterov Adam optimizer (NADAM) was used as the optimizer [31], which provides a better computational performance than stochastic gradient descent. The training of the model used a mini-batch size of 64 samples and the number of epochs was set as approximately $P/64$, where P is the size of the training population. The intermediate layers of hidden neurons contained $n/2$ nodes where n is the problem size, while the latent dimension hidden layer always contained 20 fixed hidden nodes for all of our experiments. A learning rate of 0.02 was used for both the algorithms. For VAE-EDA we have used the same model architecture as suggested by the authors in [13]. The algorithms were implemented in Python programming language using Keras Deep learning library (<https://keras.io/>).

V. RESULTS AND DISCUSSION

All results presented correspond to an average of 30 independent runs for each method. Figure 5 shows the execution time comparisons between BOA, hBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DBM-EDA and DAE-EDA for Trap 5 problems for problem sizes 25, 50, 75 and 100. We can see that both VAE-EDA-Q and VAE-EDA-Q AVS have better computational performance than VAE-EDA, BOA and CMSS-BOA. While VAE-EDA-Q and VAE-EDA-Q AVS outperforms DBM-EDA, VAE-EDA is unable to do so. The execution of time of DAE-EDA as reported by the authors turns out to be slightly lower than those of VAE-EDA-Q and VAE-EDA-Q AVS.

¹Source code is available at <https://github.com/sourodeep/vaeedaq>

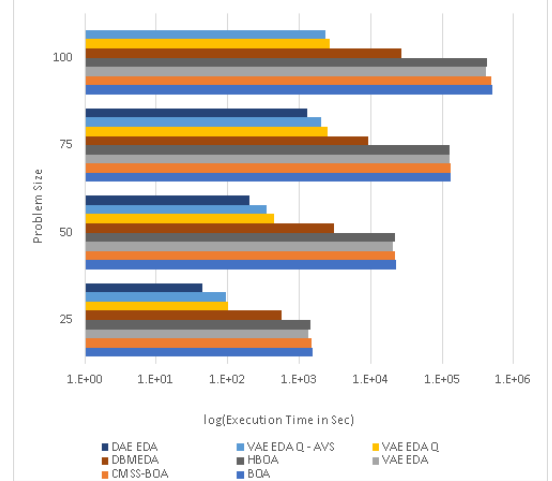


Fig. 3. Execution time for Trap 5 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DBM-EDA and DAE-EDA. VAE-EDA-Q and VAE-EDA-Q AVS have better performance than VAE-EDA, BOA and CMSS-BOA.

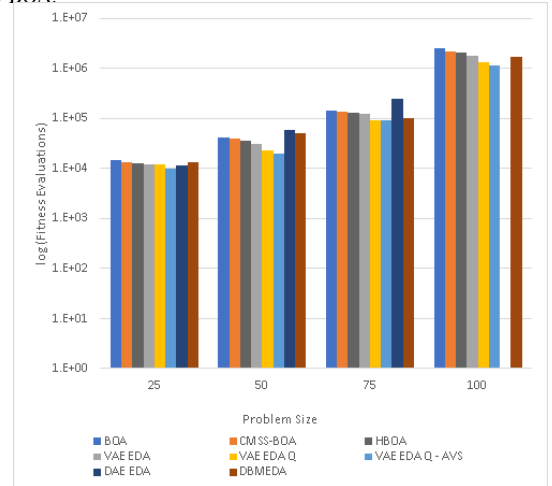


Fig. 4. Number of fitness Evaluations for Trap 5 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DBM-EDA and DAE-EDA. VAE-EDA-Q has lowest number of fitness evaluations compared to all the other algorithms

For the number of fitness evaluations (figure 4) it was observed that DAE-EDA performs more fitness evaluations than all other algorithms. VAE-EDA-Q and VAE-EDA-Q AVS performs less fitness evaluations than VAE-EDA, BOA and CMSS-BOA, with BOA being the highest. We also find that VAE-EDA-Q has better performance than VAE-EDA both in terms of execution times and fitness evaluations. The number of performance evaluations of DBM-EDA lies between that of VAE-EDA and VAE-EDA-Q.

Figures 5, 6, 7, and 8 shows similar results for VAE-EDA, VAE-EDA-Q AVS, VAE-EDA-Q, CMSS-BOA, hBOA and BOA for higher order traps of order 7 and 9 respectively. For trap 7 and trap 9 DAE-EDA and DBM-EDA could not be compared as the result for higher order traps was not provided by the authors in their original research. We find

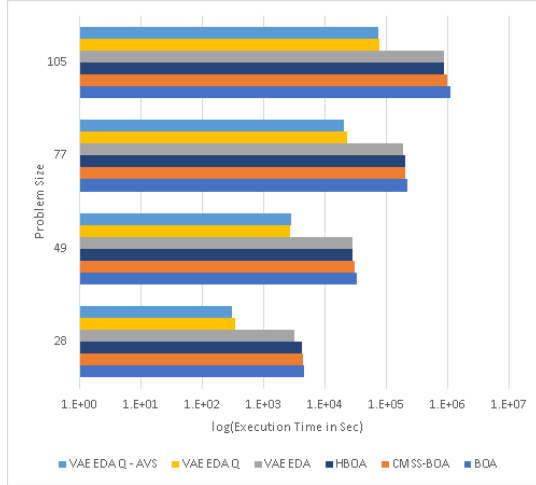


Fig. 5. Execution time for Trap 7 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q AVS and VAE-EDA-Q. VAE-EDA and VAE-EDA-Q perform better than BOA and CMSS-BOA, with VAE-EDA-Q having the lowest execution time

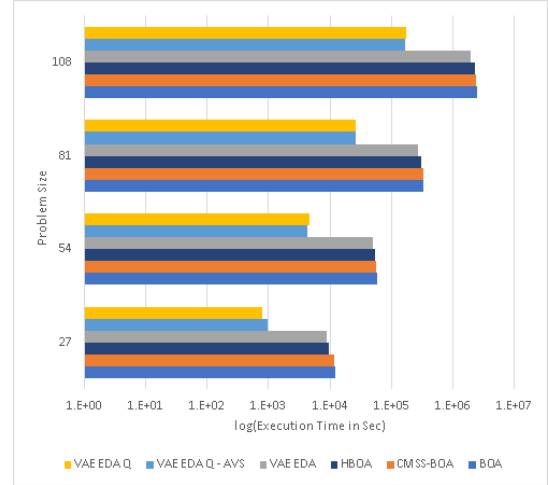


Fig. 7. Execution time for Trap 9 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q AVS and VAE-EDA-Q. VAE-EDA-Q AVS and VAE-EDA-Q perform better than VAE-EDA, BOA and CMSS-BOA, with VAE-EDA-Q having the lowest execution time

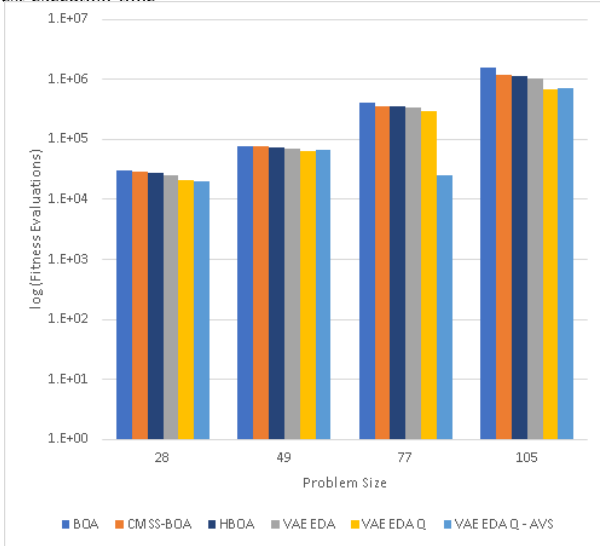


Fig. 6. Number of fitness Evaluations for Trap 7 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q AVS and VAE-EDA-Q. VAE-EDA and VAE-EDA-Q perform better than BOA and CMSS-BOA, with VAE-EDA-Q having the lowest number of fitness evaluations

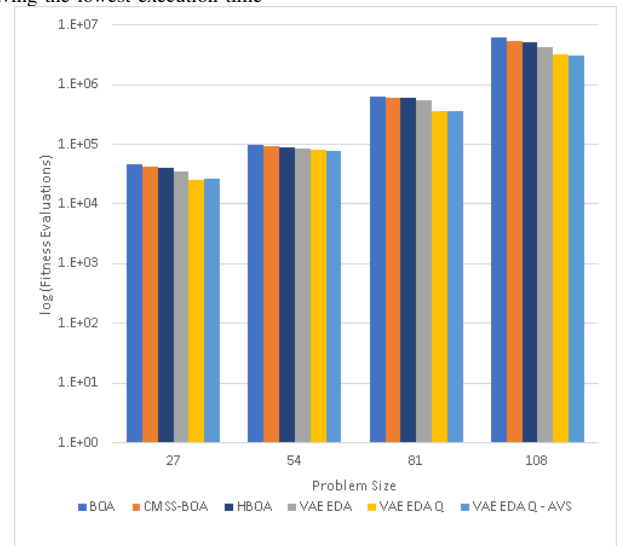


Fig. 8. Number of fitness Evaluations for Trap 9 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q AVS and VAE-EDA-Q. VAE-EDA-Q and VAE-EDA-Q AVS perform better than VAE-EDA, BOA and CMSS-BOA, with VAE-EDA-Q having the lowest number of fitness evaluations

that VAE-EDA-Q and VAE-EDA-Q AVS have approximately one- tenth the execution time requirement of BOA and perform approximately half the number of fitness evaluations as BOA.

In addition to having a better performance than other algorithms tested, VAE-EDA-Q AVS was able to solve traps of orders 11 and 13 for all 30 runs with approximately 20 to 50 times less computational time requirement than BOA and CMSS-BOA as shown in the graphs in figures 9-12. VAE-EDA and VAE-EDA-Q could not solve these problems, and no results were reported for DAE-EDA and DBM-EDA for these problems. Moreover, the success rate of BOA, hBOA and CMSS-BOA was only about 23% (approximately 7 out of 30 runs were able to find the global optimum) while the

success rate of VAE-EDA-Q AVS was 100%. These results prove that VAE-EDA-Q AVS is more capable at successfully exploring the latent space by dynamically varying the degree of exploration based on fitness.

In order to test the statistical significance of the results we performed Kruskal Wallis ANOVA [32] separately for the execution time and the number of fitness evaluations independently for each trap order and all the differences turned out to be significant (p -value < 0.05). For Trap 5 specifically we performed an additional Kruskal Wallis ANOVA test on DAE-EDA, VAE-EDA-Q and VAE-EDA-Q AVS for execution time and the result shows that the differences were not statistically significant (p -value $\bar{0}.149$ for execution time with alpha at

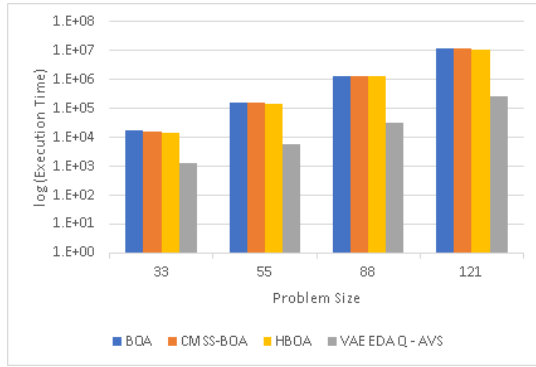


Fig. 9. Execution time for Trap 11 comparing BOA, HBOA, CMSS-BOA, and VAE-EDA-Q AVS. VAE-EDA-Q AVS has lower execution time compared to BOA and CMSS-BOA for all problem sizes.

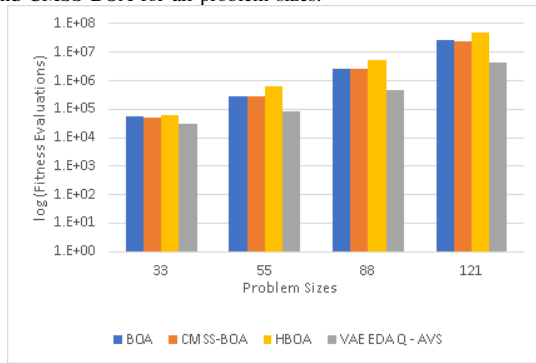


Fig. 10. Number of fitness Evaluations for Trap 11 comparing BOA, HBOA, CMSS-BOA, and VAE-EDA-Q AVS. VAE-EDA-Q AVS has lower number of fitness evaluations compared to BOA and CMSS-BOA for all problem sizes.

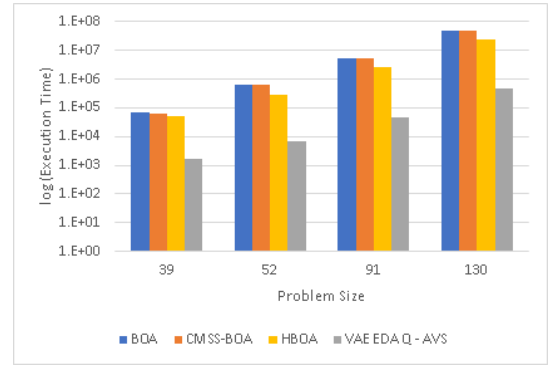


Fig. 11. Execution time for Trap 13 comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q AVS and VAE-EDA-Q. VAE-EDA-Q AVS has lower execution time compared to BOA and CMSS-BOA for all problem sizes.

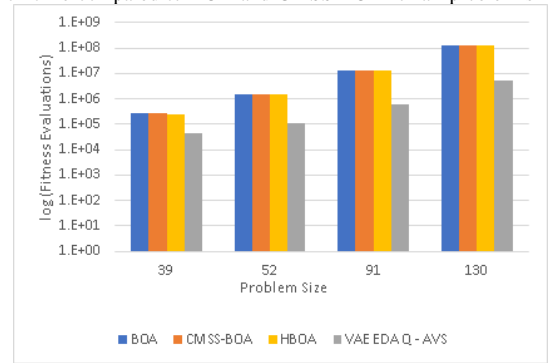


Fig. 12. Number of fitness Evaluations for Trap 13 comparing BOA, HBOA, CMSS-BOA, and VAE-EDA-Q AVS. VAE-EDA-Q AVS has lower number of fitness evaluations compared to BOA and CMSS-BOA for all problem sizes.

0.05).

Figure 13 shows the comparison of BOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DAE-EDA and DBM-EDA on NK Landscapes with problem sizes 30 and 34 and neighborhood size varying from 4 to 10. VAE-EDA-Q was found to have a slightly higher execution time than DAE-EDA, while the differences were not significant according to Kruskal Wallis ANOVA test. VAE-EDA-Q was also found to perform the smallest number of fitness evaluations (half or less) compared to all other algorithms tested for most cases.

While no results were presented for DBM-EDA and DAE-EDA for neighborhood size of 6 and above, we have performed the experiments to compare VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, BOA and CMSS-BOA on the same problem sizes but with k varying between 6 and 10. It was observed that VAE-EDA-Q and VAE-EDA-Q AVS have better performance than VAE-EDA-Q, BOA and CMSS-BOA both in terms of execution time and number of fitness evaluations.

Moreover, VAE-EDA-Q AVS was able to solve NK Landscapes with neighborhood size 8 and 10 at a better success rate than BOA. For BOA the success rate was 40% while VAE-EDA-Q AVS had a success rate of 100%. The execution time and the number of fitness evaluations for VAE-EDA-Q AVS was also better than BOA for these problems. VAE-

EDA-Q AVS was 10 - 30 times faster than BOA for NK Landscape problems. The other algorithms were either not able to solve these problems or no results were reported in past research for these algorithms for problems of this complexity. The differences in computational time observed in these comparisons were statistically significant (Kruskal Wallis ANOVA test with p -values < 0.05).

VAE-EDA-Q and VAE-EDA-Q AVS were found to have very similar execution time compared to DAE-EDA without any statistically significant differences for trap problems and NK Landscapes. VAE-EDA-Q and VAE-EDA-Q AVS were found to outperform all other algorithms including DAE-EDA in number of fitness evaluations. This can be attributed to the fact that BOA, CMSS-BOA, DBM-EDA and DAE-EDA requires larger population sizes to find global optimum and that the VAE is particularly efficient to detect and model the main dependencies. For BOA and CMSS-BOA, population size needed is calculated as $T_p = 2^k \times n^{1.05}$ [16] [33] where k is the size of the trap and n is the problem size. Hence with increasing problem difficulty (order of the trap or k) the population size needed increases exponentially. On the other hand VAE-EDA-Q and VAE-EDA-Q AVS are better at exploring latent spaces using significantly smaller population size. The maximum population size required to solve trap 9

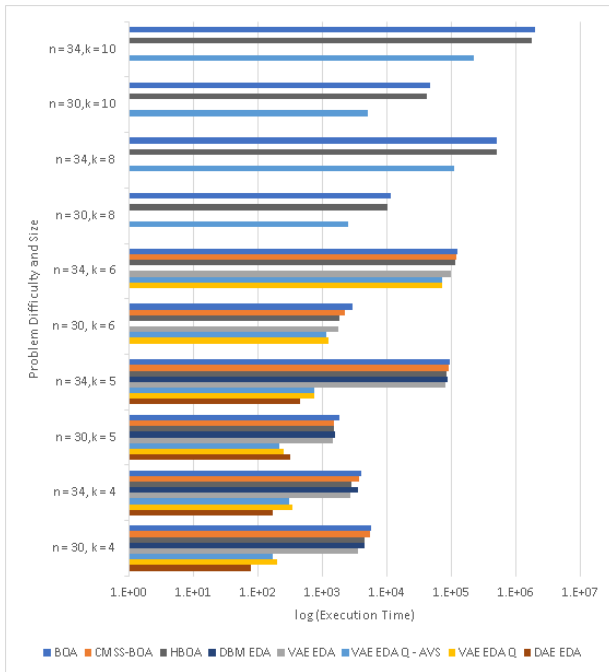


Fig. 13. Execution time for NK Landscapes comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DBM-EDA and DAE-EDA. DAE-EDA has lowest execution time upto $k = 5$. VAE-EDA-Q AVS is able to solve $K = 8$ and $K = 10$ at better performance than BOA.

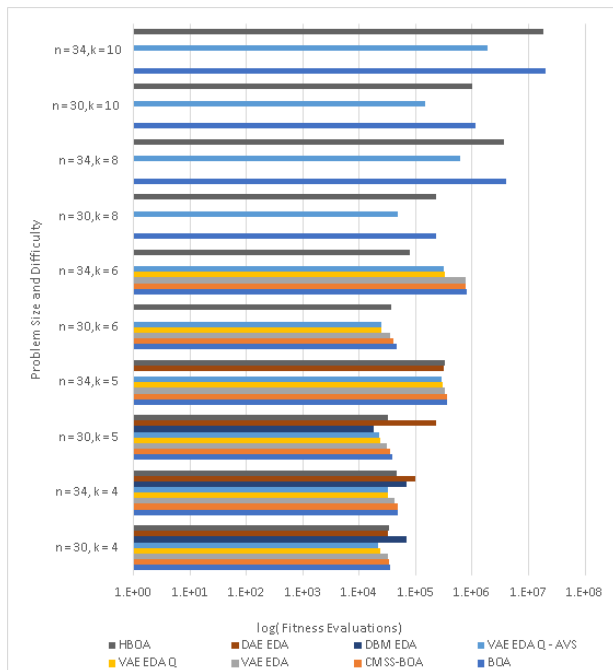


Fig. 14. Number of fitness evaluations for NK Landscapes comparing BOA, HBOA, CMSS-BOA, VAE-EDA, VAE-EDA-Q, VAE-EDA-Q AVS, DBM-EDA and DAE-EDA. VAE-EDA-Q has lowest number of fitness evaluations compared to all other algorithms. VAE-EDA-Q AVS is able to solve $K = 8$ and $K = 10$ at better performance than BOA.

for a problem of size 108 was 2400 for VAE-EDA and VAE-EDA-Q. On the other hand for DAE-EDA and DBM-EDA the population size used was up to 16000, even for trap 5 problems, while for BOA and CMSS-BOA the population size requirement was 69000 (for trap 9 problem of size 108).

We also observed that VAE-EDA-Q AVS was able to solve problems of greater difficulty than VAE-EDA, VAE-EDA-Q, DAE-EDA and DBM-EDA. These problems included Trap 11, Trap 13 and NK Landscapes with neighborhood size 8 and 10. While BOA, hBOA and CMSS-BOA were also able to solve some of these problems, their success rate was 40% or less, when VAE-EDA-Q AVS had a success rate of 100%.

VI. CONCLUSION

We have presented two novel approaches for Estimation of Distribution algorithms based on Variational Autoencoders and for exploring the use of latent variable modeling in EDAs. The first approach uses a variational autoencoder to model the population (by building its model based on a queue of population that updates at each iteration (VAE-EDA-Q). The second algorithm uses Adaptive Variance Scaling (AVS) [14] along with Queue based Variational Autoencoder model (VAE-EDA-Q AVS) to dynamically adjust the variance while sampling the latent space based on whether improvement was observed in the fitness generated. VAE-EDA-Q and VAE-EDA-Q AVS were compared to the classic Bayesian network EDA algorithms (such as BOA, hBOA and CMSS-BOA) as well as state-of-the-art algorithms (such as the standard VAE-EDA, DAE-EDA and DBM-EDA). The algorithms were tested on benchmark problems composed of Trap problems and NK Landscapes.

The results obtained were very promising in that VAE-EDA-Q required the lowest number of fitness evaluations compared to all other algorithms for all the problems tested. In terms of execution time, VAE-EDA-Q had slightly higher execution time compared to DAE-EDA (with implementation in Octave), for trap 5 and NK Landscape with neighborhood size of 5 problems, though the differences were not statistically significant. This can be attributed to differences in implementation and room for code optimization since the number of fitness evaluations performed by VAE-EDA-Q was lower than DAE-EDA. Moreover, and more importantly, the problems used for the comparisons used fitness function that are very fast to compute. For real life problems, where the fitness function can be very computationally expensive to compute, the drastic reduction in number of fitness evaluations of VAE-EDA-Q and VAE-EDA-Q AVS would lead to an important reduction in the overall computational time. In addition to having a better performance than other algorithms VAE-EDA-Q AVS is able to solve problems much more complex than the ones that can be solved by VAE-EDA, VAE-EDA-Q, DBM-EDA and DAE-EDA. While BOA, hBOA and CMSS-BOA were also able to solve some of these problems, the success rate was much lower and the number of fitness evaluation much greater (10 to 30 times more) than VAE-EDA-Q AVS.

The success of VAE-EDA-Q and VAE-EDA-Q AVS is due to the fact that they use generative modeling and are able to find solutions using about 12% of the population size required by state-of-the-art algorithms such as DAE-EDA and DBM-EDA and 3% of the population size required by Bayesian modeling algorithms such as BOA, hBOA and CMSS-BOA respectively. We have also presented results for problems of greater difficulty (such as Trap 7, Trap 9, Trap 10, Trap 13 and NK Landscapes with $K = 6$ to $K = 10$) than the ones presented for DBM-EDA and DAE-EDA, showing the robustness of our approaches with scaling complexity. We have demonstrated that VAE-EDA-Q and VAE-EDA-Q AVS outperform VAE-EDA, BOA, hBOA and CMSS-BOA in execution time and fitness evaluations for both Trap problems and NK Landscapes.

Future work involves improving upon VAE-EDA-Q AVS by dynamically changing population queue size based on the state of the iteration and balancing the amount of exploration and exploitation performed as the algorithm progresses. We expect the new algorithm to have lower execution time and also a lower number of fitness evaluations while being able to tackle even more complex problems.

REFERENCES

- [1] S. Baluja, "Population-based incremental learning. a method for integrating genetic search based function optimization and competitive learning," Carnegie-Mellon Univ Pittsburgh Pa Dept Of Computer Science, Tech. Rep., 1994.
- [2] H. Mühlenbein and G. Paass, "From recombination of genes to the estimation of distributions i. binary parameters," in *International conference on parallel problem solving from nature*. Springer, 1996, pp. 178–187.
- [3] P. Larrañaga and J. A. Lozano, *Estimation of distribution algorithms: A new tool for evolutionary computation*. Springer Science & Business Media, 2001, vol. 2.
- [4] M. Pelikan, D. E. Goldberg, and F. G. Lobo, "A survey of optimization by building and using probabilistic models," *Computational optimization and applications*, vol. 21, no. 1, pp. 5–20, 2002.
- [5] M. Hauschild and M. Pelikan, "An introduction and survey of estimation of distribution algorithms," *Swarm and evolutionary computation*, vol. 1, no. 3, pp. 111–128, 2011.
- [6] P. Smolensky, "Parallel distributed processing: Explorations in the microstructure of cognition, vol. 1. chapter information processing in dynamical systems: Foundations of harmony theory," *MIT Press, Cambridge, MA, USA*, vol. 15, p. 18, 1986.
- [7] M. Probst, F. Rothlauf, and J. Grahlf, "Scalability of using restricted boltzmann machines for combinatorial optimization," *European Journal of Operational Research*, vol. 256, no. 2, pp. 368–383, 2017.
- [8] M. Probst and F. Rothlauf, "Deep boltzmann machines in estimation of distribution algorithms for combinatorial optimization," *arXiv preprint arXiv:1509.06535*, 2015.
- [9] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [10] Y. Bengio *et al.*, "Learning deep architectures for ai," *Foundations and trends® in Machine Learning*, vol. 2, no. 1, pp. 1–127, 2009.
- [11] M. Probst, "Denoising autoencoders for fast combinatorial black box optimization," in *Proceedings of the Companion Publication of the 2015 Annual Conference on Genetic and Evolutionary Computation*. ACM, 2015, pp. 1459–1460.
- [12] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [13] U. Garcíarena, R. Santana, and A. Mendiburu, "Expanding variational autoencoders for learning and exploiting latent representations in search distributions," in *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 2018, pp. 849–856.
- [14] J. Grahlf, P. A. Bosman, and F. Rothlauf, "The correlation-triggered adaptive variance scaling idea," in *Proceedings of the 8th annual conference on Genetic and evolutionary computation*. ACM, 2006, pp. 397–404.
- [15] M. Pelikan, D. E. Goldberg, and E. Cantu-Paz, "Linkage problem, distribution estimation, and bayesian networks," *Evolutionary computation*, vol. 8, no. 3, pp. 311–340, 2000.
- [16] M. Pelikan, D. E. Goldberg, and E. Cantu-Paz, "Boa: The bayesian optimization algorithm," in *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation-Volume 1*. Morgan Kaufmann Publishers Inc., 1999, pp. 525–532.
- [17] M. Pelikan, D. E. Goldberg, and S. Tsutsui, "Hierarchical bayesian optimization algorithm: toward a new generation of evolutionary algorithms," in *SICE 2003 Annual Conference*, vol. 3. IEEE, 2003, pp. 2738–2743.
- [18] E. Salehi and R. Gras, "Efficient eda for large optimization problems via constraining the search space of models," in *Proceedings of the 13th annual conference companion on Genetic and evolutionary computation*. ACM, 2011, pp. 73–74.
- [19] I. Tsamardinos, L. E. Brown, and C. F. Aliferis, "The max-min hill-climbing bayesian network structure learning algorithm," *Machine learning*, vol. 65, no. 1, pp. 31–78, 2006.
- [20] R. Salakhutdinov and G. Hinton, "Deep boltzmann machines," in *Proceedings of the Twelfth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, D. van Dyk and M. Welling, Eds., vol. 5. Hilton Clearwater Beach Resort, Clearwater Beach, Florida USA: PMLR, 16–18 Apr 2009, pp. 448–455.
- [21] Y.-l. Boureau, Y. L. Cun *et al.*, "Sparse feature learning for deep belief networks," in *Advances in neural information processing systems*, 2008, pp. 1185–1192.
- [22] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013.
- [23] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proceedings of the 25th international conference on Machine learning*. ACM, 2008, pp. 1096–1103.
- [24] D. J. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," *arXiv preprint arXiv:1401.4082*, 2014.
- [25] C. Doersch, "Tutorial on variational autoencoders," *arXiv preprint arXiv:1606.05908*, 2016.
- [26] J. An and S. Cho, "Variational autoencoder based anomaly detection using reconstruction probability," *Special Lecture on IE*, vol. 2, pp. 1–18, 2015.
- [27] P. A. Bosman, J. Grahlf, and D. Thierens, "Enhancing the performance of maximum-likelihood gaussian edas using anticipated mean shift," in *International Conference on Parallel Problem Solving from Nature*. Springer, 2008, pp. 133–143.
- [28] K. Deb and D. E. Goldberg, "Analyzing deception in trap functions," in *Foundations of genetic algorithms*. Elsevier, 1993, vol. 2, pp. 93–108.
- [29] S. A. Kauffman and E. D. Weinberger, "The nk model of rugged fitness landscapes and its application to maturation of the immune response," *Journal of theoretical biology*, vol. 141, no. 2, pp. 211–245, 1989.
- [30] M. Pelikan, "Analysis of estimation of distribution algorithms and genetic algorithms on nk landscapes," in *Proceedings of the 10th annual conference on Genetic and evolutionary computation*. ACM, 2008, pp. 1033–1040.
- [31] T. Dozat, "Incorporating nesterov momentum into adam," 2016.
- [32] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *Journal of the American statistical Association*, vol. 47, no. 260, pp. 583–621, 1952.
- [33] G. Robin, "Study of genetic algorithms behavior for high epistasis and high dimensional deceptive functions," in *Evolutionary Computation*. InTech, 2009.

ACKNOWLEDGEMENTS

This work is supported by the NSERC grant ORGPIN 341854 and is made possible by the dedicated resource allocation 8047 of the Shared Hierarchical Academic Research Computing Network (SHARCNET, www.sharcnet.ca).